NR2DT-07: Step 7 — Migrate Logs, Tags & Drop Rules

> **Series:** NR2DT — New Relic to Dynatrace Migration Steps | **Notebook:** 7 of 10 | **Created:** April 2026 | **Last Updated:** 08/27/2026

Overview

Goal of this step: complete Wave 5 — reconfigure log forwarders to ship to Dynatrace, apply OpenPipeline parsing/drop/enrichment rules, and confirm cost-optimization patterns are working.

Procedural — see **NRLC-07** (Logs, Tags & Drop Rules) for component depth.

Table of Contents

Prerequisites

Requirement	Details
Audience	Migration lead + assigned engineer for this step

Requirement	Details
Completed	NR2DT-06 — Migrate Synthetics, SLOs, Workloads
Format	Procedural step — use as a runbook; defer to NRLC for depth
NRLC deep dives	NRLC-07 (Logs, Tags & Drops)

1. Apply OpenPipeline Configuration

```
python3 migrate.py migrate --diff --components logs,drops,parsing,tags
python3 migrate.py migrate --import-only --components logs,drops,parsing,tags
```

This applies:
- Drop rules (NRQL filters → OpenPipeline `filterOut` processors)
- Parsing rules (Grok → DPL parsers)
- Tag rules (NR entity tags → OpenPipeline enrichment rules emitting attributes on ingested data)

2. Reconfigure Log Forwarders

Reconfigure each log shipper to point at DT instead of (or in addition to) NR:

Source	Reconfigure
Filebeat / Fluent Bit	DT log ingest endpoint or OneAgent
Lambda log forwarder	DT Lambda extension or CloudWatch → Firehose → OpenPipeline
K8s log integration	DynaKube + log monitoring
Syslog	OpenPipeline syslog endpoint
HTTP/JSON	DT Generic Log Ingest API

Run dual-shipping (both NR and DT receive) for 1–2 weeks. NR is silenced after volume validation.

3. Validate Volumes and Drops

Volume parity

```
fetch logs, from:-1d
| summarize records = count(), by:{dt.system.bucket}
| sort records desc
```

Compare against NR's `SELECT count(*) FROM Log SINCE 1 day ago`. Volumes should match within ±5%.

Drop rule effectiveness

Confirm dropped patterns are being filtered:

```
fetch logs, from:-1h
| filter contains(content, "health")
| summarize count()
```

Should return 0 (or near-zero) if the drop rule is working.

Cost-optimization spot-check

```
fetch logs, from:-24h
| summarize total = count(),
    droppable = countIf(loglevel == "DEBUG" OR contains(content, "GET
/health"))
| fieldsAdd dropPercent = round((toDouble(droppable) / toDouble(total)) * 100,
decimals: 2)
```

Confirms how much volume is left to drop (target: 0% droppable after Wave 5).

4. Verify Tag Enrichment

Tags become OpenPipeline enrichment attributes attached to ingested data. Verify:

```
fetch logs, from:-1h
| filter environment == "prod"
| summarize count(), by:{environment}
```

Should return enriched data with the expected environment value.

Workload identifiers should similarly be queryable:

```
fetch logs, from:-1h
| filter workload.name == "checkout"
| summarize count()
```

5. Step Exit Criteria

G7 — Logs / Tags / Drops Migrated

- [ ] All log forwarders reconfigured to DT
- [ ] OpenPipeline parsing, drop, and enrichment rules applied
- [ ] G5 volume parity validated (±5% vs NR)
- [ ] Drop rules confirmed working (target patterns absent)
- [ ] Tag enrichment producing expected attributes on ingested data

Next step: NR2DT-08 — Validate.
